

Demand *Module*

Spatial mapping, morphological estimation, sectoral energy downscaling, and high-resolution hourly disaggregation of building energy demand at the regional scale using PostgreSQL and PostGIS.

PostgreSQL 15

PostGIS 3

GIST Indexing

Regional scope: This documentation covers the **Sicilia** implementation (schema: `sicilia`). The identical SQL logic applies to **Puglia** (schema: `puglia`) and **Calabria** (schema: `calabria`) with only schema name substitution.

FILE

LANGUAGE

Demand_Module_Sicilia/

SQL (PostgreSQL + PostGIS)

OUTPUT TABLE

TEMPORAL RESOLUTION

`sicilia_building_hourly_consumption`

Hourly (24-element arrays)

Spatial Engine Initialisation & Indexing

1.1 PostGIS Extension & Schema

`CREATE EXTENSION IF NOT EXISTS postgis` ensures the spatial engine is active. The regional schema (`sicilia`) is created if it does not exist, isolating all regional tables from other schemas.

1.2 Memory Optimisation

`SET work_mem = '1GB'` allocates generous sort/hash memory per operation. Existing idle connections of the current user are terminated to prevent lock contention during the heavy spatial joins that follow.

1.3 GIST Spatial Index Creation

R-tree (GIST) indexes are created on the geometry columns of all four spatial layers — buildings, provinces, census sections (2011 & 2021), and primary cabins — enabling sub-second Point-in-Polygon queries over millions of building footprints.

```
CREATE INDEX IF NOT EXISTS idx_sicilia_buildings_geom
ON sicilia.sicilia_buildings USING GIST(geom);
```

Spatial Mapping (Topological Integration)

Each building is assigned to its containing province, 2011 census section, 2021 census section, and primary substation area. This spatial mapping table becomes the backbone for all subsequent data joins.

2.1 Centroid Extraction for PiP Optimisation

Rather than testing full building polygon containment, **ST_Centroid()** is computed for each building and stored in a temporary table with its own GIST index. Point-in-Polygon on a point is dramatically faster than on a polygon.

```
CREATE TEMP TABLE temp_building_centroids AS
SELECT fid, ST_Centroid(geom) AS point_geom
FROM sicilia.sicilia_buildings;
```

2.2 Primary Point-in-Polygon Join

A four-way LEFT JOIN using **ST_Within()** assigns each building centroid to its province (cod_prov), 2011 section (sez2011), 2021 section (sez2021), and primary cabin area (cod_ac). **DISTINCT ON (b.fid)** prevents duplicate rows for buildings on layer boundaries.

2.3 Nearest-Neighbour Rescue Logic

Buildings with NULL assignments (boundary edge cases) are rescued using **LATERAL nearest-neighbour queries** (<-> operator). For each orphaned building, the closest polygon centroid in each layer is assigned using **ORDER BY distance LIMIT 1**.

```
LEFT JOIN LATERAL (
    SELECT cod_prov FROM public.prov_2024
    ORDER BY b.point_geom <-> geom LIMIT 1
) p_near ON TRUE
```

Master Integration & Physical Parameter Estimation

3.1 Master Table Assembly

A denormalised master table (`sicilia_buildings_integrated`) is created joining building geometry, spatial mapping, and all census indicators (E8–E20 from 2011, A8 from 2021) into a single wide table for efficient downstream processing.

3.2 Construction Year Range Estimation

Census columns **E8–E16** each count buildings in a given era within the section. The building's era is assigned as the era with the **maximum count** in its section (modal era assignment). Nine eras are mapped: Before 1919 → Post-2005.

3.3 Floor Count & Height Estimation

Columns **E17–E20** count buildings by floor-count class (1, 2, 3, 4+ floors). The modal class is assigned to each building. Final height is measured height if valid (2.5–150 m), otherwise **floors × storey height** (3.3 m pre-1980, 3.0 m modern).

Volumetric Allocation & Sectoral Classification

4.1 Energy Relevance Filter

Only buildings meeting all three criteria are processed further: **area $\geq 50 \text{ m}^2$** , **height $\geq 3.0 \text{ m}$** , valid cadastral classification (not 'Others', 'TC', 'TI').

4.2 Volume Split: Residential vs Non-Residential

$\text{volume_m3} = \text{area} \times \text{final_height}$. The residential share (`res_v`) and non-residential share (`no_res_v`) are allocated based on cadastral class (`classifica`): 'R' → residential, 'SP'/'PA' → non-residential.

4.3 Sub-sector Classification (Agr / Ind / Ser)

Non-residential volume is further split into agricultural (`agr_v`), industrial (`ind_v`), and service (`ser_v`) using `ILIKE` pattern matching on `edifc_uso` (e.g. 'Stalla', 'Capannone', 'Torre elettrica').

Downscaling, Annual Consumption & Hourly Disaggregation

5.1 Volumetric Aggregation to Census Sections

For each census section, total residential/sectoral volume is summed (WITH aggregation CTEs). Each building's share is its volume divided by section total — used as the downscaling weight.

5.2 Dwelling & Employee Downscaling

Census dwelling counts (A8, 2021) and sectoral employee counts (2011) are distributed to individual buildings proportionally by volumetric share. Active dwellings apply an occupancy correction using the 2011 empty-dwelling ratio.

```
active_dwellings = (res_v / total_res_v_sez)
                  × A8 × (1 - ratio_empty_11)
```

6.1 Annual Residential Consumption (ARERA Linkage)

$\text{cons_res_kwh_yr} = \text{active_dwellings} \times \text{yearly_kwh}$ where `yearly_kwh` comes from the ARERA Aggregator output, joined by `cod_prov`. Non-residential sectors use ENEA regional totals downscaled by employee share.

7.1 Hourly Array Generation (PL/pgSQL Loop)

A DO \$\$... FOR m IN 1..12 LOOP block iterates over each month. A temporary lookup matrix is built from the profile coefficients (output of Profile Builder). For each building, the annual kWh is multiplied by the matching coefficient to produce a **24-element real[] array** per month and day-type.

```
r_k = cons_res_kwh_yr × res_coeff -- per hour
array_agg(r_k ORDER BY hour)      -- → real[24]
```

OUTPUT TABLE: SICILIA_BUILDING_HOURLY_CONSUMPTION

building_fid	Building identifier (int8)
--------------	----------------------------

<code>year / month</code>	Reference year and month (2024, 1–12)
<code>fr_fs</code>	Day type: FR or FS
<code>res_kwh_arr</code>	24-element residential hourly consumption array (real[])
<code>ind / ser / agr _kwh_arr</code>	Sectoral 24-element arrays
<code>total_kwh_arr</code>	Sum of all sectors per hour (real[])



Note: The initial draft of this README documentation was generated with the assistance of Claude AI and subsequently reviewed, verified, and edited by the author for accuracy.